

Class 6: R Functions

Ashley Martinez (PID: A17891957)

All functions in R have at least 3 things:

-A **name**, we pick this and use it to call the function. -Input **arguments**, there can be multiple comma separated inputs to the function. -The **body**, lines of R code that do the work of the function.

Our first function:

```
add <- function(x,y=1){  
  x+y  
}
```

Let's test our function

```
add(c(1,2,3),y=10)
```

```
[1] 11 12 13
```

```
add(10,100)
```

```
[1] 110
```

A second function

Let's try something more interesting. Make a sequence generation tool.

The 'sample()' function could be useful here.

```
sample(1:10,size=3)
```

```
[1] 10 5 2
```

Change this to work with the nucleotides A,C,G,T and return 3 of them.

```
n<-c("A","C","G","T")
sample(n,size=15, replace=TRUE)
```

```
[1] "T" "C" "A" "C" "C" "A" "G" "G" "C" "G" "C" "C" "A" "G" "G"
```

Turn this snippet into function that returns a user specified length DNA sequence. Let's call it 'generate_dna()'...

```
n<-c("A","C","G","T")
sample(n,size=length(n), replace=TRUE)
```

```
[1] "T" "G" "A" "T"
```

```
generate_dna<-function(len=10,fasta=FALSE){
  n<-c("A","C","G","T")
  v<-sample(n,size=len, replace=TRUE)

  #Make a single element vector
  s<-paste(v,collapse="")

  cat("Well done you!\n")
  if (fasta){
    return(s)
  } else{
    return(v)
  }
}
```

```
generate_dna(5)
```

Well done you!

```
[1] "T" "T" "A" "T" "G"
```

```
s<-generate_dna(15)
```

Well done you!

```
s
```

```
[1] "G" "C" "G" "G" "T" "A" "T" "A" "A" "G" "A" "G" "T" "C" "C"
```

I want the option to return a single element vector with my sequence all together like this:
“GGAGTAC”

```
s
```

```
[1] "G" "C" "G" "G" "T" "A" "T" "A" "A" "G" "A" "G" "T" "C" "C"
```

```
paste(s,collapse="")
```

```
[1] "GCGGTATAAGAGTCC"
```

```
generate_dna(10,fasta=TRUE)
```

Well done you!

```
[1] "ACCTTTTGTC"
```

##A more advanced example

Make a third function that generates protein sequence of a user specified length and format.

```
generate_protein<-function(size=15,fasta=TRUE){  
  aa<-c("A","R","N","D","C","E","Q","G","H","I",  
        "L","K","M","F","P","S","T","W","Y","V","U")  
  seq<-sample(aa, size=size, replace=TRUE)  
  
  out<- if (fasta) paste(seq,collapse="") else seq  
  return(out)  
}
```

Try this out...

```
generate_protein(10)
```

```
[1] "HAEMPPQCAM"
```

Q. Generate random protein sequences between lengths 5 and 12 amino-acids.

```
generate_protein(5)
```

```
[1] "WVTGI"
```

One approach is to do this by brute force calling our function for each length 5 to 12.

Another approach is to write a “for()” loop to iterate over the input values 5 to 12.

A very useful third approach that is R specific is to use the ‘sapply()’ function.

```
seq_lengths<-6:12
for(i in seq_lengths){
  cat(">",i, "\n")
  cat(generate_protein(i))
  cat("\n")
}
```

```
> 6
UGHFER
> 7
PHIGCTC
> 8
CHILTEHW
> 9
QNECIFCTQ
> 10
WHQWMFTCQQ
> 11
SEDYPSIEHFC
> 12
VWLURTMVWIPF
```

```
sapply(6:12,generate_protein)
```

```
[1] "GUGVEK"      "GRKRRCI"      "LDPRCVMW"      "LMYEATVLY"      "DRAPMLANEE"
[6] "DNQWRQVUQMR" "TVFEIYGDUNWR"
```

Key Point: Writing functions in R is doable but not the easiest thing in the world starting with our working snippet of code then using LLM tools to improve and generalize your function code is a productive approach.